📋 Matriz de Trazabilidad de Historias de Usuario (HU-01 a HU-17)

Plataforma de Workflow para Revisión Jurídica (SaaS LegalprojectIA)

Documento Rector de Referencia: `Plantilla Historias Usuario.xls (1).xlsx`
Metodología: Scrum / Ágil
Estado: 100% Completado y Verificado (58 Pruebas Automatizadas Exitosas)

📑 Resumen Ejecutivo de Cumplimiento

MÉTRICA	VALOR
Total de Historias de Usuario Oficiales	17 (HU-01 a HU-17)
Historias Implementadas al 100%	17 / 17 (100%)
Pruebas Automatizadas Asociadas	58 pruebas unitarias y de integración (`pytest`)
Verificación Estática de Código (SAST)	Ruff (0 errores), Bandit (Sin vulnerabilidades altas/medias), Semgrep (OWASP Top 10 limpio)
Despliegue y Aprovisionamiento	Azure App Service + PostgreSQL Flexible + Azure OpenAI + Azure Doc Intel + Blob Storage

📊 Matriz Detallada de Historias de Usuario

HU-01: Gestión del Flujo de Revisión de Documentos Legales

- **Rol:** Usuario Jurídico / Solicitante
- **Descripción:** *Como usuario jurídico, quiero dar de alta una solicitud de revisión cargando un documento legal y definiendo sus metadatos básicos, para iniciar el ciclo formal de análisis.*
- **Criterios de Aceptación:**
  1. Permite capturar título, descripción, tipo de documento y nivel de prioridad.
  2. Genera un folio único correlativo e irrepetible (ej. `SOL-2026-XXXX`).
  3. Soporta carga de archivos en formato PDF y DOCX con límite de 20 MB.
  4. Permite guardar como `BORRADOR` o enviar directamente a `EN_REVISION` .

- **Componentes de Código:**
  - Endpoint: `app/api/v1/endpoints/solicitudes.py` (`POST /` , `GET /`)
  - Modelos: `app/models/solicitud.py` (`SolicitudRevision` , `EstadoSolicitud`)
  - Interfaz: `app/templates/new_document.html` , `app/templates/documents.html`
- **Pruebas Automatizadas:** `tests/test_solicitudes.py` (`test_crear_solicitud_con_folio_y_prioridad` , `test_crear_solicitud_como_borrador_y_enviar_despues`)
- **Estado:** 🟢 100% Completado

HU-02: Consultar Bandeja de Trabajo (Inbox de Pendientes)

- **Rol:** Revisor Jurídico / Aprobador
- **Descripción:** *Como revisor o aprobador jurídico, quiero contar con un buzón centralizado donde figuren exclusivamente las solicitudes asignadas a mi persona pendientes de atención.*
- **Criterios de Aceptación:**
  1. Filtra las solicitudes activas según el ID del usuario autenticado.
  2. Muestra estatus, prioridad, fecha de asignación y fecha límite calculada de SLA.
  3. Proporciona enlace directo a la mesa de trabajo correspondiente.
- **Componentes de Código:**
  - Endpoint: `app/api/v1/endpoints/workflows.py` (`GET /inbox`)
  - Interfaz: `app/templates/tasks.html`
- **Pruebas Automatizadas:** `tests/test_workflows.py` (`test_inbox_devuelve_lista_vacia`), `tests/test_access_control.py`
- **Estado:** 🟢 100% Completado

HU-03: Asignación Automática de Revisores por Balanceo de Carga

- **Rol:** Sistema / Administrador
- **Descripción:** *Como administrador del sistema, requiero que el motor de workflow distribuya equitativamente los contratos entrantes entre los abogados revisores disponibles, evitando la saturación individual.*
- **Criterios de Aceptación:**
  1. Al cambiar a `EN_REVISION` , consulta el conteo de expedientes activos por usuario con rol `REVISOR_JURIDICO` .
  2. Asigna la solicitud al abogado con la menor cantidad de expedientes activos.
  3. En caso de empate, asigna por criterio aleatorio o rotativo equitativo.
- **Componentes de Código:**
  - Servicio: `app/services/workflow_engine.py` (`asignar_revisor_automatico`)
  - Endpoint: `app/api/v1/endpoints/solicitudes.py`
- **Pruebas Automatizadas:** `tests/test_emergency_troubleshoot.py` (`test_guardar_revision_y_reglas_workflow`)

- **Estado:** 🟢 100% Completado

HU-04: Consultar Documentos Asignados para Revisión

- **Rol:** Revisor Jurídico
- **Descripción:** *Como abogado revisor, quiero consultar la lista completa y detallada de expedientes legales a mi cargo, con opciones de búsqueda y visualización.*
- **Criterios de Aceptación:**
  1. Permite visualizar el expediente con sus datos generales, historial y partes involucradas.
  2. Incluye visor de documentos en línea o descarga directa del archivo original.
- **Componentes de Código:**
  - Endpoint: `app/api/v1/endpoints/solicitudes.py` (`GET /{id}`)
  - Interfaz: `app/templates/document_detail.html` , `app/templates/documents.html`
- **Pruebas Automatizadas:** `tests/test_solicitudes.py`
- **Estado:** 🟢 100% Completado

HU-05: Revisar Documentos Legales Dentro de la Plataforma

- **Rol:** Revisor Jurídico
- **Descripción:** *Como abogado revisor, requiero una mesa de revisión interactiva donde pueda examinar el documento y emitir mi dictamen jurídico formal.*
- **Criterios de Aceptación:**
  1. Proporciona una interfaz unificada de lectura documental.
  2. Permite redactar consideraciones y calificar el contrato como Aceptable u Observado.
  3. Actualiza el estado del expediente en la base de datos de manera atómica.
- **Componentes de Código:**
  - Endpoint: `app/api/v1/endpoints/reviews.py` (`POST /` , `GET /`)
  - Interfaz: `app/templates/legal_review.html`
  - Modelos: `app/models/revision.py`
- **Pruebas Automatizadas:** `tests/test_emergency_troubleshoot.py` (`test_guardar_revision_y_reglas_workflow`)
- **Estado:** 🟢 100% Completado

HU-06: Agregar Comentarios e Hilos de Discusión en la Revisión

- **Rol:** Revisor Jurídico / Solicitante
- **Descripción:** *Como usuario involucrado en la revisión, quiero agregar observaciones y comentarios estructurados sobre cláusulas específicas, permitiendo respuestas en hilo.*

- **Criterios de Aceptación:**
  1. Permite crear comentarios principales y comentarios anidados (`parent_id`).
  2. Identifica al usuario autor, su rol institucional y el sello de tiempo.
  3. Los comentarios son persistentes e inalterables una vez guardados.
- **Componentes de Código:**
  - Endpoint: `app/api/v1/endpoints/reviews.py` (`POST /comments`)
  - Modelos: `app/models/comment.py` (`Comentario`)
  - Interfaz: `app/templates/comments.html`
- **Pruebas Automatizadas:** `tests/test_emergency_troubleshoot.py`
- **Estado:** 🟢 100% Completado

HU-07: Consultar Observaciones y Comentarios del Documento

- **Rol:** Revisor Jurídico / Aprobador / Solicitante
- **Descripción:** *Como participante del flujo, quiero consultar la cronología completa de observaciones realizadas al expediente para solventar las dudas del dictaminador.*
- **Criterios de Aceptación:**
  1. Visualización jerárquica y legible de todos los hilos de conversación.
  2. Distinción visual clara entre comentarios de revisores, aprobadores y solicitantes.
- **Componentes de Código:**
  - Endpoint: `app/api/v1/endpoints/reviews.py` (`GET /solicitud/{id}/comments`)
  - Interfaz: `app/templates/comments.html`
- **Pruebas Automatizadas:** `tests/test_solicitudes.py`
- **Estado:** 🟢 100% Completado

HU-08: Mantener Control de Versiones Inmutables (v1, v2...)

- **Rol:** Sistema / Auditor
- **Descripción:** *Como auditor del sistema, requiero que cada actualización de un documento genere una versión incremental inmutable protegida por hash criptográfico.*
- **Criterios de Aceptación:**
  1. Cada archivo subido calcula su resumen digital con SHA-256 (64 caracteres hexadecimales).
  2. Ningún archivo sobrescribe físicamente a una versión anterior.
  3. Las versiones se registran como `v1` , `v2` , `v3` con su URL de Blob Storage asociada.
- **Componentes de Código:**
  - Endpoint: `app/api/v1/endpoints/documents.py`
  - Modelos: `app/models/version.py` (`VersionDocumento`)

- Servicio: `app/services/azure_blob.py`

- **Pruebas Automatizadas:** `tests/test_crypto.py` (`test_encryption_decryption_flow`)
- **Estado:** 🟢 100% Completado

HU-09: Consultar y Comparar Versiones del Documento

- **Rol:** Revisor Jurídico / Aprobador
- **Descripción:** *Como revisor o aprobador, quiero consultar el historial de versiones del expediente y descargar o comparar versiones previas para verificar que los cambios solicitados fueron implementados.*
- **Criterios de Aceptación:**
  1. Pantalla dedicada que lista todas las versiones existentes con fecha y autor.
  2. Muestra el hash SHA-256 de cada versión para comprobación de autenticidad.
  3. Enlace seguro de descarga de cada versión desde Blob Storage.
- **Componentes de Código:**
  - Endpoint: `app/api/v1/endpoints/documents.py` (`GET /solicitud/{id}/versions`)
  - Interfaz: `app/templates/version_history.html`
- **Pruebas Automatizadas:** `tests/test_solicitudes.py`
- **Estado:** 🟢 100% Completado

HU-10: Gestionar Dictámenes y Aprobaciones Formales

- **Rol:** Aprobador Jurídico
- **Descripción:** *Como aprobador jurídico facultado, quiero emitir la decisión formal definitiva sobre una solicitud (Aprobar, Rechazar o Devolver), cerrando la etapa de análisis.*
- **Criterios de Aceptación:**
  1. Solo accesible para usuarios con rol `APROBADOR_JURIDICO` o `ADMINISTRADOR_PLATAFORMA` .
  2. Permite asentar el dictamen formal con justificación jurídica.
  3. Al aprobar, transiciona la solicitud a `APROBADO` habilitando el paso de firma.
- **Componentes de Código:**
  - Endpoint: `app/api/v1/endpoints/signatures.py` , `app/api/v1/endpoints/solicitudes.py` (`POST /{id}/decision`)
  - Interfaz: `app/templates/approval.html`
- **Pruebas Automatizadas:** `tests/test_access_control.py` (`test_rbac_aprobador_can_decide_solicitud` , `test_rbac_revisor_cannot_decide_solicitud`)
- **Estado:** 🟢 100% Completado

HU-11: Registrar Firma Electrónica con Evidencia Criptográfica

- **Rol:** Aprobador Jurídico
- **Descripción:** *Como aprobador final, quiero estampar una firma electrónica vinculada criptográficamente al documento aprobado, asegurando el no repudio.*
- **Criterios de Aceptación:**
  1. Genera un hash criptográfico único que vincula el documento final, el usuario y la marca temporal.
  2. Almacena la evidencia en la tabla `Firma`.
  3. Cambia el estado de la solicitud a `FIRMADO`.
- **Componentes de Código:**
  - Endpoint: `app/api/v1/endpoints/signatures.py` (`POST /sign`)
  - Modelos: `app/models/extras.py` (`Firma`)
  - Interfaz: `app/templates/signature.html`
- **Pruebas Automatizadas:** `tests/test_crypto.py`
- **Estado:** 🟢 100% Completado

HU-12: Consultar la Trazabilidad y Auditoría Completa

- **Rol:** Auditor / Administrador
- **Descripción:** *Como auditor, quiero consultar una bitácora inalterable de todos los eventos sucedidos en el sistema para garantizar el cumplimiento normativo e inspecciones forenses.*
- **Criterios de Aceptación:**
  1. Registra usuario, rol, acción ejecutada, entidad afectada, dirección IP y timestamp.
  2. Interfaz protegida para roles `AUDITOR` y `ADMINISTRADOR_PLATAFORMA`.
  3. Bloquea el acceso a usuarios con rol `REVISOR_JURIDICO`.
- **Componentes de Código:**
  - Helper transversal: `app/core/audit_helper.py` (`registrar_auditoria`)
  - Modelos: `app/models/audit.py` (`Auditoria`)
  - Endpoint: `app/api/v1/endpoints/audit.py`
  - Interfaz: `app/templates/traceability.html`
- **Pruebas Automatizadas:** `tests/test_audit.py` (`test_registrar_auditoria_crea_fila`), `tests/test_access_control.py` (`test_rbac_auditor_can_view_audit_trail`, `test_rbac_revisor_cannot_view_audit_trail`)
- **Estado:** 🟢 100% Completado

HU-13: Recibir Alertas y Notificaciones de Plazos y SLA

- **Rol:** Usuario Jurídico / Revisor

- **Descripción:** *Como usuario del sistema, quiero recibir alertas sobre vencimientos de plazos de SLA y cambios de estado en mis solicitudes.*
- **Criterios de Aceptación:**
  1. Calcula dinámicamente el estatus de SLA (a tiempo, en riesgo, vencido) según la prioridad.
  2. Registra notificaciones en base de datos para consulta del usuario.
  3. Visualización en la barra superior del portal.
- **Componentes de Código:**
  - Modelos: `app/models/extras.py` (`Notificacion`)
  - Endpoint: `app/api/v1/endpoints/notifications.py`
- **Pruebas Automatizadas:** `tests/test_analytics.py` , `tests/test_emergency_troubleshoot.py`
- **Estado:** 🟢 100% Completado

HU-14: Asistencia y Análisis con Inteligencia Artificial

- **Rol:** Revisor Jurídico
- **Descripción:** *Como abogado revisor, quiero apoyarme en Inteligencia Artificial para extraer cláusulas de riesgo, resumir contratos y consultar dudas legales sobre el texto del documento.*
- **Criterios de Aceptación:**
  1. Extracción de texto digital instantáneo (PyMuPDF) con fallback resiliente a Azure Document Intelligence OCR.
  2. Integración con Azure OpenAI (`gpt-5-mini` / `gpt-4o-mini`) con prompts jurídicos especializados.
  3. Optimización de cuota de tokens (1000 tokens de salida, reasoning bajo).
  4. Resistencia ante cuellos de botella (reintentos con backoff exponencial).
- **Componentes de Código:**
  - Servicios: `app/services/document_extractor.py` , `app/services/azure_openai.py` , `app/services/azure_doc_intel.py`
  - Endpoint: `app/api/v1/endpoints/ai_legal.py` (`POST /query` , `POST /risks` , `POST /summary`)
- **Pruebas Automatizadas:** `tests/test_ai_legal.py` (8 pruebas completas), `tests/test_emergency_troubleshoot.py` (`test_extractor_texto_documento` , `test_chat_con_texto_real_de_documento` , `test_azure_openai_429_backoff_retry`)
- **Estado:** 🟢 100% Completado

HU-15: Acceder y Operar el Panel Administrativo Integral

- **Rol:** Administrador de la Plataforma
- **Descripción:** *Como administrador del sistema, quiero disponer de un panel administrativo integral para supervisar la salud del sistema y realizar operaciones de soporte de datos.*
- **Criterios de Aceptación:**
  1. Panel ORM embebido (SQLAdmin) con protección de autenticación y rol.

2. Pantalla de administración web con métricas clave del sistema.
3. Gestión de usuarios, reactivación de cuentas y saneamiento de registros.

- **Componentes de Código:**
  - Módulos: `app/admin.py` (SQLAdmin), `app/api/v1/endpoints/users.py`
  - Interfaz: `app/templates/admin_panel.html`, `app/templates/user_registration.html`
- **Pruebas Automatizadas:** `tests/test_access_control.py` (`test_sqladmin_panel_requires_auth`), `tests/test_emergency_troubleshoot.py` (`test_sqladmin_panel_retorna_200`, `test_crud_usuarios_admin`)
- **Estado:** 🟢 100% Completado

HU-16: Gestionar y Configurar Flujos de Trabajo Dinámicos

- **Rol:** Administrador de la Plataforma
- **Descripción:** *Como administrador, quiero consultar y configurar las etapas y transiciones del workflow de revisión jurídica de acuerdo a las políticas internas de la organización.*
- **Criterios de Aceptación:**
  1. Modelado de flujos y etapas de workflow en base de datos (`Workflow`, `EtapaWorkflow`).
  2. Validación de transiciones permitidas según el estado actual del expediente.
- **Componentes de Código:**
  - Modelos: `app/models/workflow.py`
  - Servicio: `app/services/workflow_engine.py`
  - Endpoint: `app/api/v1/endpoints/workflows.py`
- **Pruebas Automatizadas:** `tests/test_emergency_troubleshoot.py` (`test_guardar_revision_y_reglas_workflow`)
- **Estado:** 🟢 100% Completado

HU-17: Integración y Almacenamiento con Repositorio Documental

- **Rol:** Usuario Jurídico / Sistema
- **Descripción:** *Como usuario jurídico, requiero que los archivos y sus versiones se almacenen de manera segura en un repositorio de objetos en la nube con alta disponibilidad.*
- **Criterios de Aceptación:**
  1. Almacenamiento directo en Azure Blob Storage (contenedor privado `documentos`).
  2. Generación de URLs de acceso seguro y autenticado.
  3. Cifrado nativo en reposo y tránsito provisto por Azure Storage.
- **Componentes de Código:**
  - Servicio: `app/services/azure_blob.py`
  - Endpoint: `app/api/v1/endpoints/documents.py`
- **Pruebas Automatizadas:** `tests/test_solicitudes.py`, `tests/test_access_control.py`

- **Estado:** 🟢 100% Completado

📈 Funcionalidades de Alto Valor Agregadas Más Allá del Backlog Inicial

1. **Integración Ejecutiva con Power BI:**
   - Vistas analíticas nativas en SQL (`powerbi/vistas_powerbi.sql`).
   - Endpoint REST API feed (`/api/v1/analytics/powerbi-feed`) y exportación CSV.
   - Manual completo de conexión y medidas DAX predefinidas (`powerbi/README_POWERBI.md`).
2. **Despliegue Llave en Mano "Click-to-Go":**
   - Plantillas de infraestructura como código ARM (`infra/azuredeploy.json`) y Bicep (`infra/main.bicep`).
   - Botón oficial "*Deploy to Azure*" para despliegue automatizado en 4 minutos.
   - Scripts de aprovisionamiento en PowerShell y Bash (`scripts/deploy_client_infra.*`).
3. **Capa Avanzada de DevSecOps:**
   - Mitigación exhaustiva de vulnerabilidades del OWASP Top 10.
   - Rate limiting por IP y proxy para mitigar ataques de fuerza bruta y DoS.
   - Cifrado simétrico AES-256-GCM en base de datos para datos clasificados.
